

Google Workspace Integration Plan

Prepared: 2026-05-30

Scope

K-CUBE needs two Google Workspace integrations:

1. Google Single Sign-On for user authentication.
2. Google Calendar API sync for K-CUBE events.

This plan is backend-first because authentication and calendar tokens must be verified and controlled by the Node.js backend, not trusted directly from the browser.

Integration 1: Google Single Sign-On

Current State

The backend already has:

```
POST /api/v1/auth/google
```

But the current implementation accepts raw `google_id`, `email`, and `full_name` from the frontend. That is acceptable only as a placeholder. Production must verify Google-issued credentials.

Recommended Flow

```
Next.js frontend
-> Google Identity Services
-> receives ID token or authorization code
-> POST /api/v1/auth/google

Node.js backend
-> verifies token or exchanges code with Google
-> reads verified Google claims
-> creates/links K-CUBE user
-> stores auth identity
-> returns K-CUBE JWT pair
```

Backend Verification Rules

- Verify token signature and issuer.
- Verify audience equals `GOOGLE_CLIENT_ID`.
- Require `email_verified = true`.
- Use Google `sub` as stable provider ID.
- Link existing user by verified email only when safe.
- Create `auth_identities` row with provider `google`.
- Award welcome points only once.

Data Storage

Use existing/proposed tables:

- users
- auth_identities
- session_logs
- point_transactions

Recommended identity row:

```
{
  "provider": "google",
  "provider_user_id": "google-sub-claim",
  "email": "member@gmail.com",
  "verified": true
}
```

Environment

```
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_AUTH_REDIRECT_URI=
GOOGLE_ALLOWED_WORKSPACE_DOMAIN=
```

GOOGLE_ALLOWED_WORKSPACE_DOMAIN should stay empty if normal Gmail users are allowed.

Integration 2: Google Calendar API

Business Goal

K-CUBE admins should create or update an event once in K-CUBE, then sync that event to Google Calendar. The frontend event pages should continue reading K-CUBE's database, not Google Calendar directly.

Recommended Ownership

K-CUBE database is the source of truth.

Google Calendar is the external publishing/scheduling channel.

This avoids the UI depending on Google's API availability and gives K-CUBE its own RSVP, points, K-Food, learning, and admin workflows.

Phase 1 Calendar Mode

Use one shared K-CUBE calendar controlled by an admin OAuth connection.

Best for:

- Fast delivery.
- One official event calendar.
- Lower Workspace admin complexity.

Phase 2 Calendar Mode

Use Google Workspace service account with domain-wide delegation only when K-CUBE needs domain-level calendar access, such as managing calendars across organization users or rooms without each user's consent.

Best for:

- Internal team calendars.

- Room/resource calendars.
- Centralized Workspace administration.

Calendar Scopes

Minimum event sync scope:

```
https://www.googleapis.com/auth/calendar.events
```

Use broader calendar scope only if K-CUBE needs to create calendars, list calendars, or manage calendar settings.

Sync Operations

K-CUBE action	Calendar action
Publish event	Insert Google Calendar event
Update title/time/location	Update Google Calendar event
Cancel event	Cancel/delete Google Calendar event based on admin setting
RSVP user	Optional attendee update; default should stay K-CUBE-only
Admin check-in	Award event points in K-CUBE, no Calendar change required

Required Backend Services

```
backend/src/services/googleAuthService.ts
backend/src/services/googleCalendarService.ts
backend/src/services/eventService.ts
backend/src/routes/events.ts
backend/src/routes/googleWorkspace.ts
```

Required API Endpoints

```
POST /api/v1/auth/google
GET /api/v1/events
GET /api/v1/events/:slug
POST /api/v1/events/:id/rsvp
POST /api/v1/events/:id/check-in
POST /api/v1/admin/events
PATCH /api/v1/admin/events/:id
POST /api/v1/admin/events/:id/sync/google-calendar
GET /api/v1/admin/google-calendar/connections
POST /api/v1/admin/google-calendar/connections
POST /api/v1/admin/google-calendar/sync
```

Required Database Additions

See `docs/2026-05-30/DATABASE_ERD_AND_SCHEMA.md` for SQL definitions:

- `platform_events`
- `platform_event_rsmps`
- `google_calendar_connections`

- `google_calendar_sync_jobs`

Failure Handling

- If Google sync fails, keep K-CUBE event saved with `sync_status = failed`.
- Store error details in `google_calendar_sync_jobs`.
- Admin can retry sync.
- Frontend should display K-CUBE event data even when Google sync is delayed.

Google Cloud Setup Checklist

1. Create/select Google Cloud project. 2. Configure OAuth consent screen. 3. Create OAuth 2.0 web client. 4. Add authorized JavaScript origins:

- `http://localhost:3000`
- production frontend domain

5. Add authorized redirect URIs if using authorization code redirect flow:

- `http://localhost:3000/auth/google/callback`
- production callback URL

6. Enable Google Calendar API. 7. Store client ID/secret in backend environment variables. 8. For Workspace domain-wide delegation, create service account and approve scopes in Google Admin Console.

Security Notes

- Keep Google client secret and service account private key backend-only.
- Never expose service account keys in Next.js `NEXT_PUBLIC_*` variables.
- Encrypt stored OAuth refresh tokens if admin OAuth is used.
- Restrict admin calendar connection endpoints to `role = admin`.
- Log calendar connection changes to `admin_audit_logs`.

Source References

- Google Identity Services and OAuth for web apps: <https://developers.google.com/identity/oauth2/web/guides/how-user-authz-works>
- Google OAuth 2.0 web server flow: <https://developers.google.com/identity/protocols/oauth2/web-server>
- Google Calendar events API: <https://developers.google.com/workspace/calendar/api/v3/reference/events/insert>
- Google Calendar sync guidance: <https://developers.google.com/workspace/calendar/api/guides/sync>
- Google Workspace domain-wide delegation: <https://support.google.com/a/answer/162106>